Lab 5 Report

Machine Learning for Hardware Design

Authors: Zach Dixon, Jongwoo Kim

Team Number: 06

1. Six MAC units (3.3)

We made 4-bit and 2-bit versions of the staged and pipelined MAC, so we have six MACs with the 8-bit ones from Lab 3. Both MACs already had a C_DATA_WIDTH generic, so all versions use the same VHDL with a different generic value.

MAC	Bits	LUT	FF	DSP	WNS (ns)	Fmax (MHz)	Dynamic power (W)	Static power (W)
Staged	8	118	41	0	0.834	240.0	0.025	0.105
Staged	4	76	41	0	1.100	256.4	0.023	0.105
Staged	2	53	41	0	1.283	269.0	0.023	0.105
Pipelined	8	41	138	1	0.511	222.8	0.016	0.105
Pipelined	4	49	122	1	0.578	226.1	0.015	0.105
Pipelined	2	56	114	1	0.721	233.7	0.015	0.105

Table 1: Each MAC built alone at a 200 MHz constraint. Fmax = 1 / (5 ns - WNS).

The staged MAC gets smaller with fewer bits (118 to 53 LUTs). The pipelined MAC does not, because its multiplier sits in one DSP block at every width.

The six XSA files are in hw/xsa, next to the timing, utilization and power reports of each full system. The reports of each MAC built alone are in hw/staged_mac/vivado and hw/piped_mac/vivado.

2. Full inference on the MAC units (3.4)

computeAccelerated sends every multiply-accumulate of the convolution and dense layers to the MAC unit. We ran test image 0 through each of the six MACs on the ZedBoard and compared every layer output with the software quantized result, byte by byte. All 13 layers match for all six MACs.

MAC	Bits	All 13 layers match	Software on the ARM core (ms)	Through the MAC unit (ms)
Staged	8	Yes	1,353	29,073
Staged	4	Yes	1,353	29,080
Staged	2	Yes	1,348	28,303
Pipelined	8	Yes	1,353	29,074
Pipelined	4	Yes	1,353	29,080
Pipelined	2	Yes	1,348	28,303

Table 2: One inference of image 0 on the ZedBoard. The logs are in util/board_logs.

3. Power, performance and area (4)

Throughput and latency (4.1)

T is the clock period and N is the number of MACs in one group (one conv or dense output). We assume no back-pressure.

MAC	Minimum latency of one group	Maximum throughput
Staged	N * T	N / ((N + 1) * T)
Pipelined	(N + 3) * T	1 / T

The staged MAC needs one more cycle after each group to send the result. The pipelined MAC has three more register stages, but groups can follow each other with no gap.

MAC	N	Latency (ns)	Throughput (M MACs/s)
Staged	1	10	50.0
Staged	5	50	83.3
Staged	100	1,000	99.0
Pipelined	1	40	100
Pipelined	5	80	100
Pipelined	100	1,030	100

Table 3: Example values at T = 10 ns (the 100 MHz system clock).

Power, area and energy of one inference (4.2)

The peak power and the area of each MAC are in Table 1. One inference of the Toy Model needs 131,595,776 MACs in 310,728 groups. For the energy we use the 200 MHz clock, because the power reports were made at that clock.

Staged, 8-bit:

- Cycles = 131,595,776 MACs + 310,728 groups = 131,906,504
- Time = 131,906,504 / 200 MHz = 0.660 s
- Power = 0.105 W static + 0.025 W dynamic = 0.130 W
- Energy = 0.130 W x 0.660 s = 85.7 mJ

MAC	Time (s)	Energy at 8 / 4 / 2 bits (mJ)
Staged	0.660	85.7 / 84.4 / 84.4
Pipelined	0.658	79.6 / 79.0 / 79.0

Table 4: Estimated energy of one inference. The pipelined MAC needs 131,595,779 cycles (the MACs plus 3 cycles to fill the pipeline).

More than one PE (4.3)

How do power, energy, area and performance scale?

With P MAC units in parallel, throughput and area grow by P and the time for one inference drops by P. Dynamic power also grows by P, but static power is paid only once. So the dynamic energy per inference stays the same and the static energy gets smaller.

PEs	LUT	Time (s)	Power (W)	Energy (mJ)
1	118	0.660	0.130	85.7
3	354	0.220	0.180	39.6
5	590	0.132	0.230	30.3
100	11,800	0.0066	2.605	17.2

Table 5: The staged 8-bit MAC with more PEs. The energy cannot go below the dynamic part, 0.025 W x 0.660 s = 16.5 mJ.

Do these values make sense? (4.4)

Do they over or underestimate?

They are far too optimistic for our board. The estimated time is 1.32 s at 100 MHz and we measured 29.07 s, which is 22 times longer. One MAC takes about 0.22 us on the board, or 22 clock cycles instead of 1.

What are we not taking into account?

- Data movement. The ARM core writes one word at a time to the FIFO through memory-mapped I/O and waits for each result.
- The rest of the system. The full system uses 1.683 W, and 1.529 W of that is the ARM processor. The MAC uses 2 mW or less. One real inference costs about 1.683 W x 29.07 s = 48.9 J, not 85.7 mJ.
- The FIFO. It needs 764 LUTs and 704 flip-flops, more than six times the staged 8-bit MAC.
- The power reports are estimates without real signal activity.

Execution time of all implementations (4.5)

The measured times are in Table 2. The staged and pipelined MACs differ by 1 ms at 8 bits, and the 8-bit and 4-bit runs are the same. The quantized model on the ARM core alone takes 1.35 s, so going through the MAC is 21.5 times slower.

Why are the 2-bit runs 2.7% faster?

This does not come from the MAC. The staged and pipelined 2-bit runs take exactly the same time, and the gain is the same in every layer, about 6 ns per word. One word takes 21.5 clock cycles in the 8-bit and 4-bit builds and 20.9 in the 2-bit build. We compared the compiled code of the 4-bit and 2-bit builds: the loop that writes to the FIFO has the same number of instructions and only the constants differ. The 8-bit build has one instruction fewer and is not faster. So the

amount of code is not the reason. We think it comes from how the timing of the ARM core lines up with the 100 MHz clock each time a word crosses to the FIFO, but we did not test this.

Layer	MACs	Software (ms)	MAC unit (ms)	us per MAC	Share of MAC time
conv1	8,640,000	119	1,973	0.228	6.8%
conv2	80,281,600	674	17,660	0.220	60.7%
conv3	12,460,032	102	2,762	0.222	9.5%
conv4	21,233,664	287	4,682	0.220	16.1%
conv5	3,686,400	49	813	0.221	2.8%
conv6	4,718,592	109	1,045	0.221	3.6%
dense1	524,288	13	127	0.242	0.4%
dense2	51,200	0	11	0.215	0.0%

Table 6: Per-layer times for the staged 8-bit MAC. Max pooling, flatten and softmax take less than 1 ms.

Why are the speeds the same?

The time per MAC is the same in every layer, so the run time only follows the number of MACs. The time goes to moving words between the ARM core and the FIFO, and every version moves the same number of words.

Effect of a variable-precision MAC (4.6)

- Area: our variable MAC uses 842 LUTs and 968 registers, more than any fixed-width MAC in Table 1. Sharing its 2-bit multiplier bricks across modes does not make this implementation smaller.
- Power and energy: estimated system power is 1.690 W versus 1.683 W for staged 8-bit. The PS7 accounts for 1.529 W, so the energy benefit mainly comes from shorter run time.
- Performance: packing multiple operand pairs in one FIFO data word reduces ARM-to-FIFO writes. One word holds 2 pairs at 8 bits, 4 at 4 bits, or 8 at 2 bits.
- Accuracy: the Lab 4 whole-model results show that low precision cannot be applied uniformly without a large accuracy loss. Section 5.3 selects widths layer by layer with a separate calibrated quantizer.

4. Variable-precision MAC (5)

Choice and format (5.1–5.2)

Which method did we choose?

Spatial accumulation. In our system the ARM core sends each FIFO word by memory-mapped I/O, so putting more multiply-accumulates into each word can reduce the number of writes. A header at the start of each packet selects the operand width: code 0 is 8 bits, 1 is 4 bits, and 2 is 2 bits.

Each 32-bit data word holds 16 bits of weights above 16 bits of activations. The MAC computes 2, 4, or 8 operand pairs per word at 8, 4, or 2 bits. TLAST returns the 32-bit accumulated sum and clears the accumulator for the next packet.

Precision of each layer (5.3)

Layer	Bits	Layer	Bits	Layer	Bits
conv1	8	conv2	4	conv3	8
conv4	4	conv5	4	conv6	4
dense1	4	dense2	8		

Table 7: The runtime width of each convolution and dense layer.

We started with calibrated 8-bit quantization in every layer. We lowered the layer that saved the most FIFO words while top-1 and top-10 accuracy stayed within one percentage point of that baseline on 800 validation images. The last 200 validation images were used to choose calibration ranges. No layer was lowered to 2 bits because the accuracy loss was too large.

Implementation and checks (5.4)

The same 2-bit multiplier bricks are reused at all three widths. The brick products and each level of the adder tree are registered before the final 32-bit accumulation. The software writes the mode in each packet header, so the width can change between layers without changing the hardware.

The VHDL testbench checked 132,432 packets with zero errors. Standalone post-route timing met the 200 MHz constraint with WNS +0.747 ns. The exported XSA contains the variable_mac module and a 32-bit connection to the FIFO; system WNS was +2.367 ns. On the ZedBoard, exhaustive operand-pair self-tests passed at 8, 4 and 2 bits, and all 13 inference layers matched the quantized software model.

Comparison with fixed precision (5.5)

MAC	LUT	FF	DSP	Fmax MHz	Power W
Staged 8	118	41	0	240.0	0.130
Staged 4	76	41	0	256.4	0.128
Staged 2	53	41	0	269.0	0.128
Pipelined 8	41	138	1	222.8	0.121
Pipelined 4	49	122	1	226.1	0.120
Pipelined 2	56	114	1	233.7	0.120
Variable mixed	842	968	0	235.1	0.144

Table 8: Standalone post-route MAC reports at a 200 MHz constraint. Power is a Vivado estimate.

Board design	MAC time ms	System power W	Estimated energy J
Staged 8	29,073	1.683	48.93
Staged 4	29,080	1.683	48.94
Staged 2	28,303	1.684	47.66
Pipelined 8	29,074	1.685	48.99
Pipelined 4	29,080	1.685	49.00
Pipelined 2	28,303	1.686	47.72
Variable mixed	11,546	1.690	19.51

Table 9: One image-0 inference on the ZedBoard. Energy = measured MAC time × estimated system power; it is not a direct power measurement.

The mixed-width run performs 131,595,776 MACs in 38,244,352 data words and 310,728 packets. The fixed-width designs send 131,595,776 data words. Packing cuts data words to 29.1% and reduces measured MAC time from 29.073 s (staged 8-bit) to 11.546 s, a 2.52-fold speedup. The variable run remains 8.54 times slower than its 1.352 s ARM-only software run because the ARM still writes each remaining word to the FIFO.

For a compute-only estimate at 200 MHz, the variable stream needs about 38,244,352 data cycles + 310,728 header cycles + 7 pipeline-fill cycles = 38,555,087 cycles, or 0.193 s. At its estimated 0.144 W, that is 27.8 mJ. Staged 8-bit needs 131,906,504 cycles, or 0.660 s and 85.7 mJ. These ideal values omit the data movement that dominates the board time.

Calibrated design	Top-1 %	Top-10 %	Images
Uniform 8-bit	24.125	60.500	800
Uniform 4-bit	21.625	59.750	800
Uniform 2-bit	3.375	17.250	800
Mixed 8/4/8/4/4/4/4/8	24.000	60.625	800

Table 10: Same calibrated quantization method for every row: images 800–999 calibrate, images 0–799 evaluate.

The mixed model retains nearly the same accuracy as calibrated uniform 8-bit while sending fewer FIFO words. Lab 4's fixed-width board runs use its required scale and zero-point method, so the difference between Lab 4 fixed accuracy and this calibrated mixed accuracy cannot be attributed only to variable precision. The mixed model scored 23.5% top-1 and 60.6% top-10 on all 1,000 images, but that count includes the 200 calibration images; Table 10 is the separate evaluation set.

What drove and limited each benefit?

Packing reduced FIFO traffic and therefore board time and estimated system energy. The shared brick logic and registered adder tree increased MAC area and standalone power. System power changed little because the PS7 dominates. Packet headers and one-at-a-time ARM writes limited the speedup. The 2-bit mode is supported in hardware but not selected for this model because of its accuracy loss.